

并发控制报告

概述

本报告记录了航空售票系统后端（`app.py`）在并发场景下存在的竞态条件漏洞，以及对应的修复方案与验证逻辑。

一、问题背景

1.1 原始代码模式（TOCTOU 竞态）

修复前，购票、退票、改签三个接口均采用以下不安全模式：

```
1. SELECT  → 读取当前状态（余票 / 订单状态）
2. 业务判断 → 校验是否满足条件
3. UPDATE  → 执行写操作
```

步骤 1 和步骤 3 之间存在时间窗口。在这个窗口内，另一个并发请求可以读到相同的旧值并同样通过校验，最终两个请求都执行写操作，产生数据不一致。

这类漏洞称为 **TOCTOU**（**Time of Check to Time of Use**，**检查时间与使用时间竞态**）。

二、漏洞详情

2.1 购票接口 `/api/buy_ticket`

漏洞场景（超卖）：

时序	请求 A（用户甲）	请求 B（用户乙）
T1	SELECT 余票 = 1, 校验通过	—
T2	—	SELECT 余票 = 1, 校验通过
T3	INSERT 订单, UPDATE remain = remain - 1 → 0	—
T4	—	INSERT 订单, UPDATE remain = remain - 1 → -1

结果：余票为 -1，产生超卖。

2.2 退票接口 `/api/refund_ticket`

漏洞场景（重复归还座位）：

时序	请求 A	请求 B
T1	SELECT 订单状态 = 已支付, 校验通过	—
T2	—	SELECT 订单状态 = 已支付, 校验通过
T3	UPDATE 状态 = 已退票, remain + 1	—
T4	—	UPDATE 状态 = 已退票 (再次), remain 再 + 1

结果：座位归还两次，余票凭空增加；订单状态被写为已退票两次（幂等表象但副作用不幂等）。

2.3 改签接口 /api/change_ticket

漏洞场景 1（旧航班座位多次归还）： 与退票类似，并发改签导致同一旧航班座位归还多次。

漏洞场景 2（新航班超卖）： 多个改签请求并发锁定同一目标航班，新航班校验通过后均执行扣减，产生超卖。

三、修复方案

3.1 技术选型

采用 **MySQL InnoDB 悲观行锁 (SELECT ... FOR UPDATE)** + **原子条件 UPDATE** 双重防线：

防线	机制	作用
第一道	SELECT ... FOR UPDATE	对目标行加排他锁，同一行的并发请求在锁释放前阻塞排队，消除时间窗口
第二道	UPDATE ... WHERE remain > 0 / WHERE status NOT IN (...)	即使锁粒度在极端情况下失效，原子条件也能保证不写出非法值
兜底检查	cur.rowcount == 0	检测 UPDATE 是否实际命中行，若为 0 则回滚并返回业务错误

前提条件：

- MySQL 存储引擎为 **InnoDB**（默认，支持行级锁）
- pymysql 连接默认 **autocommit=False**，锁在 **commit** 或 **rollback** 时自动释放
- FOR UPDATE** 必须在事务内执行（pymysql 默认满足）

3.2 购票接口修复

```
# 1. FOR UPDATE 锁住目标航班实例行（串行化同一航班的并发购票）
cur.execute(
    "SELECT first_remain, economy_remain FROM flight_instance "
    "WHERE flight_no=%s AND fly_date=%s FOR UPDATE",
    (flight_no, fly_date)
)
```

```

inst = cur.fetchone()
if not inst:
    conn.rollback()
    return jsonify({"code": 400, "msg": "该航班实例不存在或已取消"})

remain = inst["first_remain"] if cabin_level == "头等舱" else
inst["economy_remain"]
if remain <= 0:
    conn.rollback()
    return jsonify({"code": 400, "msg": f"{cabin_level}已售罄，余票不足"})

# 2. 原子扣减（第二道防线：WHERE remain > 0）
cur.execute(
    "UPDATE flight_instance SET economy_remain=economy_remain-1 "
    "WHERE flight_no=%s AND fly_date=%s AND economy_remain > 0",
    (flight_no, fly_date)
)

# 3. rowcount 兜底
if cur.rowcount == 0:
    conn.rollback()
    return jsonify({"code": 400, "msg": f"{cabin_level}已售罄，请重新查询"})

```

数据流：

```

请求 A —— FOR UPDATE —— 获得锁 —— 扣减 —— commit —— 释放锁
请求 B —— FOR UPDATE —— 等待锁 —— 获得锁 —— 读到 remain=0 —— |
                                                              |
                                                              └── rollback, 返回"已售罄"

```

3.3 退票接口修复

```

# 1. FOR UPDATE 锁住订单行（防止并发重复退同一张票）
cur.execute(
    "SELECT * FROM ticket_record WHERE ticket_id=%s FOR UPDATE",
    (ticket_id,)
)
ticket = cur.fetchone()
if not ticket or ticket["ticket_status"] in ["已退票", "已完成"]:
    conn.rollback()
    return jsonify({"code": 400, "msg": "无法退票（订单不存在或已处理）"})

# 2. 状态条件原子更新（第二道防线）
cur.execute(
    "UPDATE ticket_record SET ticket_status='已退票' "
    "WHERE ticket_id=%s AND ticket_status NOT IN ('已退票', '已完成')",
    (ticket_id,)
)

```

```
if cur.rowcount == 0:
    conn.rollback()
    return jsonify({"code": 400, "msg": "退票失败，订单状态已变更，请刷新后重试"})

# 3. 归还座位（此时锁已保证唯一执行）
cur.execute(
    "UPDATE flight_instance SET economy_remain=economy_remain+1 "
    "WHERE flight_no=%s AND fly_date=%s",
    (flight_no, fly_date)
)
```

3.4 改签接口修复

改签需要同时锁两行，因此需要注意**锁顺序一致性**，避免死锁：

- 始终先锁订单行（`ticket_record`）
- 再锁目标航班实例行（`flight_instance`）

```
# 1. 锁原订单行
cur.execute(
    "SELECT * FROM ticket_record WHERE ticket_id=%s FOR UPDATE",
    (ticket_id,)
)

# 2. 锁目标航班实例行
cur.execute(
    "SELECT first_remain, economy_remain FROM flight_instance "
    "WHERE flight_no=%s AND fly_date=%s FOR UPDATE",
    (new_flight_no, new_fly_date)
)

# 3. 归还旧航班座位
# 4. 原子扣减新航班座位（WHERE remain > 0）
# 5. 原子更新旧订单状态（WHERE status NOT IN (...)）
# 6. 插入新订单
```

死锁预防： 所有改签请求的加锁顺序固定为 `ticket_record` → `flight_instance`，不会产生循环等待。

四、修复效果对比

场景	修复前	修复后
100 人同时抢 1 张票	多人成功，余票为负	仅 1 人成功，余票最小为 0
同一张票并发退票	座位归还多次，余票虚增	仅第一次成功，后续返回业务错误
同一张票并发改签	旧航班多次归还，新航班超卖	串行处理，数据一致
购票后立即退票（并发）	可能产生负票	锁保证操作互斥，数据正确

五、局限性说明

1. **单实例限制：** `SELECT ... FOR UPDATE` 是数据库级行锁，在单 MySQL 实例下有效。若未来扩展为主从集群或分布式数据库，需要配合分布式锁（如 Redis Redlock）或数据库事务协调器。
2. **锁等待超时：** MySQL 默认 `innodb_lock_wait_timeout = 50`（秒）。高并发峰值下大量请求长时间排队可能触发锁等待超时，建议根据业务场景调小超时值，并在前端做友好的重试提示。
3. **性能影响：** 悲观锁会导致同一航班的并发购票请求串行化，吞吐量受单行锁制约。对于极高并发场景（如热门航班秒杀），可升级为基于 Redis 的令牌桶或分布式队列方案。
4. **autocommit 依赖：** 修复方案依赖 pymysql 默认的 `autocommit=False`。若代码中任何地方显式设置了 `autocommit=True`，`FOR UPDATE` 将不生效（因为没有事务包裹）。当前代码无此问题。

六、文件变更记录

文件	修改位置	修改说明
ds-aviation-backend/app.py	buy_ticket() 函数	加 <code>FOR UPDATE</code> ，原子扣减，rowcount 兜底
ds-aviation-backend/app.py	refund_ticket() 函数	加 <code>FOR UPDATE</code> 锁订单行，状态条件原子更新
ds-aviation-backend/app.py	change_ticket() 函数	加 <code>FOR UPDATE</code> 锁订单行+目标实例行，原子扣减，rowcount 兜底